

Spring Core

IoC · DI · Container · Configuration · Scope · Lifecycle

Topics Covered in This Module

The Real Problem Spring Solves · IoC Principle · BeanFactory vs ApplicationContext · All 3 Configuration Styles · Stereotype Annotations · Bean Scopes · Bean Lifecycle · Constructor / Setter / Field Injection · @PostConstruct / @PreDestroy · Conditional Beans · Application Events · Full Project Lab · 20 Interview Q&As



1. The Real Problem Spring Solves — Enterprise Context

Before writing a single annotation, you must deeply understand the problem Spring was built to solve. In a real enterprise system, classes have many dependencies — and managing those dependencies manually creates a cascade of serious problems. Let's walk through a realistic ShopFlow scenario.

1.1 The Scenario — User Registration Service

When a user registers on ShopFlow, the system must perform six distinct operations in sequence:

1. Validate email uniqueness — check the database
2. Encode the password — BCrypt hashing
3. Save user to the database — persist the record
4. Send a welcome email — via SMTP or SendGrid
5. Publish an audit log — record who registered and when
6. Trigger an analytics event — for business dashboards

In traditional Java, a developer instinctively writes this:

```
UserService.java — Tightly Coupled (Traditional Java)
// ——— TRADITIONAL JAVA — The Wrong Way ———
// File: UserService.java
public class UserService {

    // Each dependency is created inside the class using 'new'
    // UserService is in FULL CONTROL of creating its own dependencies
    private UserRepository repo = new UserRepositoryImpl();
    private PasswordEncoder encoder = new BCryptPasswordEncoder(12);
    private EmailService email = new SmtplibEmailService();
    private AuditService audit = new AuditService();
    private AnalyticsService analytics = new GoogleAnalyticsService();

    public void registerUser(String emailAddr, String password) {
        if (repo.existsByEmail(emailAddr)) {
            throw new RuntimeException("Email already taken");
        }
        String hashed = encoder.encode(password);
        User user = new User(emailAddr, hashed);
        repo.save(user);
        email.sendWelcome(emailAddr); // hardwired to SMTP!
        audit.log("USER_REGISTERED", emailAddr);
        analytics.track("registration", emailAddr);
    }
}
```

Enterprise Problems — Why This Fails at Scale

✗ TIGHT COUPLING: UserService is permanently bound to SmtplibEmailService, UserRepositoryImpl, etc.

To swap SMTP → SendGrid you must change UserService source code. Every change = risk.

✗ IMPOSSIBLE TO UNIT TEST: Running UserServiceTest also runs the real DB and real SMTP server.

Tests fail in CI because there is no SMTP in the build pipeline.

✗ VIOLATES SOLID — Dependency Inversion Principle: High-level UserService depends on low-level SmtplibEmailService directly, instead of on the EmailService abstraction.

✗ CONFIGURATION SCATTERED: The BCrypt strength (12), SMTP host, DB URL are all hardwired

in class files. Changing prod config requires code changes and recompilation.

✗ CANNOT HAVE MULTIPLE CONFIGURATIONS: You cannot run the same UserService with ConsoleEmailService in tests and SendGridEmailService in production without code changes.

✗ NO LIFECYCLE MANAGEMENT: Who is responsible for closing the DB connection? Flushing logs?

In large systems, resource leaks accumulate silently.

2. Inversion of Control (IoC)

2.1 Definition

IoC (Inversion of Control) is a design principle where object creation and dependency management are delegated to an external container. The 'control' of constructing and wiring objects is inverted — taken away from the class and given to the container.

Without IoC (Traditional)	With IoC (Spring)
Class creates its own dependencies: <code>new EmailService()</code>	Class declares what it needs: <code>private final EmailService emailService</code>
Class knows HOW dependencies are built	Class only knows WHAT it needs — container decides HOW
Changing a dep requires changing the class	Swapping a dep = change the container config, not the class
Testing requires real infrastructure	Testing: pass a mock to the constructor — no Spring needed
Configuration hardcoded in source	Configuration externalised — YAML, env vars, profiles

2.2 The Hotel Room Service Analogy

Analogy — Hotel Room Service

Scenario: You check into a hotel. You are hungry. In the 'traditional Java' world, you would go to the market, buy vegetables, find a kitchen, cook, plate the food, and bring it to your room yourself. In the 'Spring IoC' world, you pick up the phone and say 'I need breakfast'. The hotel (the container) figures everything else out.

Spring Container = Hotel. Your Class = Hotel Guest. Dependencies = Breakfast. You declare WHAT you need (`@Autowired` / constructor parameter). Spring decides HOW it is created, which implementation, which configuration. You don't know which chef cooked your food (`SmtEmailService` or `SendGridEmailService`). You don't care — you just needed breakfast (an `EmailService`).

2.3 The Same UserService — With IoC Applied

Now let's rewrite the tight-coupled `UserService` using IoC. Notice: no 'new' keyword anywhere inside `UserService`.

UserService.java — IoC Applied

```
// — SPRING IoC — The Correct Way —————
// File: UserService.java
@Service // Tells Spring: manage this class as a bean
public class UserService {

    // Dependencies DECLARED, not created
    // UserService says 'I need these' — Spring provides them
    private final UserRepository userRepository;
```

```

    private final PasswordEncoder passwordEncoder;
    private final EmailService emailService;    // interface - not
    SmtplibEmailService!
    private final AuditService auditService;
    private final AnalyticsService analyticsService;

    // Spring calls this constructor and injects all five dependencies
    public UserService(UserRepository userRepository,
                       PasswordEncoder passwordEncoder,
                       EmailService emailService,
                       AuditService auditService,
                       AnalyticsService analyticsService) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.emailService = emailService;
        this.auditService = auditService;
        this.analyticsService = analyticsService;
    }

    public void registerUser(String email, String password) {
        if (userRepository.existsByEmail(email)) {
            throw new EmailAlreadyExistsException(email);
        }
        String hashed = passwordEncoder.encode(password);
        User user = userRepository.save(new User(email, hashed));
        emailService.sendWelcome(email);    // works with ANY EmailService impl
        auditService.log("USER_REGISTERED", email);
        analyticsService.track("registration", email);
    }
}

// In tests: pass MockEmailService, MockAuditService via constructor
// In prod:   Spring injects SendGridEmailService, DatabaseAuditService
// UserService source code NEVER changes regardless of which impl is used

```

✓ What IoC Gives Us

- ✓ UserService depends on interfaces (EmailService), not implementations (SmtplibEmailService).
- ✓ Swap any implementation by changing container config — UserService source untouched.
- ✓ Unit test with: new UserService(mockRepo, mockEncoder, mockEmail, mockAudit, mockAnalytics).
- ✓ No Spring context needed in unit tests — fast, isolated, predictable.
- ✓ All configuration lives in YAML/env vars — not scattered through class files.

? Interview Question: What is Inversion of Control (IoC)? How is it different from Dependency Injection (DI)?

- ✓ IoC is the PRINCIPLE: control of object creation is transferred from the class to an external container. Instead of 'new EmailService()', you declare 'I need an EmailService' and the container provides it. DI is the MECHANISM Spring uses to implement IoC — it injects (provides) declared dependencies into your classes at runtime. Analogy: IoC = the concept of room service (you don't cook). DI = the waiter physically bringing the food to your table. IoC is the 'what', DI is the 'how'.

3. The Spring IoC Container

The Spring IoC Container is the core engine of the Spring Framework. It reads configuration metadata, creates beans based on that metadata, wires them together, and manages their entire lifecycle.

3.1 What the Container Does

- Creates beans — instantiates objects based on configuration
- Injects dependencies — wires beans together (constructor, setter, or field)
- Manages lifecycle — calls `@PostConstruct` on init, `@PreDestroy` on shutdown
- Applies AOP proxies — wraps beans for `@Transactional`, `@Async`, `@Cacheable`, etc.
- Manages scopes — singleton (shared), prototype (new per request), request, session
- Publishes events — `ApplicationEvent` system for loose coupling within a service

3.2 Three Ways to Configure the Container

The container reads configuration from three possible sources — you must understand all three because real enterprise systems mix them:

Style	How It Works	When Used
XML-Based	Bean definitions in <code>applicationContext.xml</code>	Legacy enterprise systems, existing codebases
Java <code>@Configuration</code>	<code>@Configuration</code> class with <code>@Bean</code> methods	Infrastructure beans: <code>DataSource</code> , <code>Kafka</code> , <code>Security</code>
Annotation (Component Scan)	<code>@Component</code> / <code>@Service</code> / <code>@Repository</code> auto-detected	Business beans: <code>Services</code> , <code>Repositories</code> , <code>Controllers</code>

4. BeanFactory vs ApplicationContext

Spring provides two container interfaces arranged in a hierarchy. BeanFactory is the root; ApplicationContext extends it and adds enterprise capabilities. Understanding the distinction is a classic interview topic.

4.1 BeanFactory — The Basic Container

Package: org.springframework.beans.factory.BeanFactory

BeanFactory — Lazy Initialisation

```
// ——— BeanFactory — Basic programmatic usage ———  
// Loads bean definitions from XML and provides getBean()  
BeanFactory factory = new XmlBeanFactory(  
    new ClassPathResource("applicationContext.xml")  
);  
  
// Bean is created HERE — only when getBean() is called (LAZY)  
// If no one ever calls getBean('userService'), UserService is never instantiated  
UserService service = factory.getBean(UserService.class);
```

BeanFactory Characteristics

- LAZY initialisation — beans created only when first requested via getBean().
- No i18n support (MessageSource).
- No event publishing (ApplicationEvent system).
- BeanPostProcessor and BeanFactoryPostProcessor must be registered manually.
- Lightweight — historically used in resource-constrained environments (Android, IoT).
- You will almost NEVER use BeanFactory directly in modern Spring or Spring Boot.

4.2 ApplicationContext — The Enterprise Container

Package: org.springframework.context.ApplicationContext — extends BeanFactory.

ApplicationContext — Implementations and Bean Retrieval

```
// ——— ApplicationContext — Three common implementations ———  
  
// 1. Loads beans from an XML file on the classpath (legacy)  
ApplicationContext ctx =  
    new ClassPathXmlApplicationContext("applicationContext.xml");  
  
// 2. Loads beans from @Configuration classes (modern standalone apps)  
ApplicationContext ctx =  
    new AnnotationConfigApplicationContext(AppConfig.class);  
  
// 3. Spring Boot creates this automatically — you never write it manually  
// SpringApplication.run() internally creates:  
// AnnotationConfigServletWebServerApplicationContext (for web apps)  
ConfigurableApplicationContext ctx =  
    SpringApplication.run(ShopflowUserServiceApplication.class, args);
```

```
// Retrieving beans - three ways:
UserService us1 = ctx.getBean(UserService.class);           // by type
// (preferred)
UserService us2 = ctx.getBean("userService", UserService.class); // by name + type
UserService us3 = (UserService) ctx.getBean("userService");    // by name (avoid)
```

Feature	BeanFactory	ApplicationContext
Bean Initialisation	LAZY — only on getBean()	EAGER — all singletons at startup (fail-fast)
i18n / MessageSource	✗ Not supported	✓ Full support
Event Publishing	✗ Not supported	✓ Built-in ApplicationEvent system
BeanPostProcessor	✗ Manual registration	✓ Auto-detected
AOP / @Transactional	✗ Not automatic	✓ Seamless
Environment / Profiles	✗ Not supported	✓ @Profile, @Value, PropertySource
Used in practice	Extremely rare	Always — Boot creates it automatically

? Interview Question: What is the difference between BeanFactory and ApplicationContext? Which should I use?

✓ BeanFactory is the basic IoC container — lazy bean creation, minimal features. ApplicationContext extends it with eager singleton initialisation (fail-fast on misconfiguration), i18n, event publishing, AOP, and automatic BeanPostProcessor detection. In any modern Spring or Spring Boot application, you ALWAYS use ApplicationContext — Spring Boot creates it for you via SpringApplication.run(). BeanFactory is only relevant when studying Spring internals or working on extremely resource-constrained systems. Key interview point: ApplicationContext IS-A BeanFactory (implements the interface) but adds full enterprise features on top.

5. Configuration Styles — All Three in Depth

Spring supports three ways to tell the container what beans to create and how to wire them. Enterprise systems mix all three — XML for legacy integration, `@Configuration` for infrastructure, annotations for business logic. You must be comfortable reading and writing all three.

5.1 XML-Based Configuration — Legacy But Essential to Know

XML configuration was Spring's original approach (pre-2.5). Every bean and its dependencies are declared in an XML file. You will encounter this in legacy enterprise systems, banking applications, and older e-commerce platforms.

applicationContext.xml — Complete Example

```
<!-- ——— applicationContext.xml — ShopFlow User Service (XML Style) ——— -->
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- EmailService bean: setter injection for host and port -->
    <bean id="emailService"
          class="com.shopflow.service.SmtEmailService">
        <property name="host" value="smtp.gmail.com"/>
        <property name="port" value="587"/>
    </bean>

    <!-- PasswordEncoder: constructor injection (strength = 12) -->
    <bean id="passwordEncoder"
          class="org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder">
        <constructor-arg value="12"/>
    </bean>

    <!-- UserRepository: prototype scope (new instance per request) -->
    <bean id="userRepository"
          class="com.shopflow.repository.UserRepositoryImpl"
          scope="prototype"/>

    <!-- UserService: constructor injection referencing other beans -->
    <bean id="userService"
          class="com.shopflow.service.UserService"
          init-method="onStartup"
          destroy-method="onShutdown">
        <constructor-arg ref="emailService"/>
        <constructor-arg ref="passwordEncoder"/>
        <constructor-arg ref="userRepository"/>
    </bean>

</beans>
```

Loading XML Config Programmatically

```
// ——— Loading XML Application Context in Java ———
public class XmlConfigDemo {
    public static void main(String[] args) {
        ApplicationContext ctx =
```

```

    new ClassPathXmlApplicationContext("applicationContext.xml");

    UserService userService = ctx.getBean("userService", UserService.class);
    userService.registerUser("alice@shopflow.com", "Password@123");

    // Close context to trigger destroy-method callbacks
    ((ClassPathXmlApplicationContext) ctx).close();
}

```

XML Tag / Attribute	What It Does
<bean id='x' class='com.X'>	Registers a bean named 'x' of type com.X
<property name='host' value='smtp.gmail.com'/>	Calls setHost('smtp.gmail.com') — setter injection
<constructor-arg value='12'/>	Passes 12 to the constructor — constructor injection
<constructor-arg ref='emailService'/>	Passes the 'emailService' bean to the constructor
scope='prototype'	New instance every time the bean is requested
init-method='onStartup'	Calls onStartup() after all deps are injected
destroy-method='onShutdown'	Calls onShutdown() when context is closed

💡 When XML Config Is Still Useful

- Legacy enterprise codebases — often thousands of lines of existing XML config.
- Third-party framework integration — some older frameworks only support XML.
- Externalising large configuration — when non-developers need to modify bean wiring.
- When class source cannot be modified — cannot add `@Component` to a library class.
- Migration projects — gradually replacing XML with annotations, section by section.

5.2 Java-Based Configuration — `@Configuration` + `@Bean`

Java configuration (Spring 3.0+) is the modern replacement for XML. You write type-safe Java instead of XML strings. This is the preferred approach for infrastructure beans — `DataSource`, `Kafka`, `Security`, `JWT`, `Redis` — anything that requires complex construction logic or uses third-party classes.

5.2.1 `@Configuration` — The Container Factory Class

📄 AppConfig.java — `@Configuration` Basics

```

// — @Configuration explained —
// Spring applies CGLIB subclassing to @Configuration classes.
// This intercepts @Bean method calls so that calling passwordEncoder()
// twice returns the SAME singleton bean, not two new objects.
// IMPORTANT: @Configuration classes cannot be final (CGLIB cannot subclass final).

@Configuration
public class AppConfig {
    // This class is a factory for Spring beans
    // Spring registers every @Bean method's return value as a bean
}

```

5.2.2 @Bean — Defining Beans Explicitly

SecurityConfig.java — @Bean Definitions

```
// — File: SecurityConfig.java — Infrastructure Beans —————
@Configuration
public class SecurityConfig {

    // Method name = default bean name: 'passwordEncoder'
    // Return type = bean type: PasswordEncoder
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12); // third-party class → use @Bean
    }

    // @Bean with explicit name
    @Bean(name = "jwtHelper")
    public JwtUtil jwtUtil(@Value("${jwt.secret}") String secret,
                           @Value("${jwt.expiry-ms:900000}") long expiryMs) {
        return new JwtUtil(secret, expiryMs);
    }

    // @Bean with dependencies: Spring resolves params from the context
    @Bean
    public UserService userService(UserRepository userRepository,
                                   PasswordEncoder passwordEncoder) {
        // Spring injects userRepository and passwordEncoder automatically
        return new UserService(userRepository, passwordEncoder);
    }

    // @Bean calling another @Bean method — safe due to CGLIB proxy
    // This does NOT create a new PasswordEncoder — returns the cached singleton
    @Bean
    public AuditPasswordHelper auditHelper() {
        return new AuditPasswordHelper(passwordEncoder()); // returns cached bean
    }
}
```

DatabaseConfig.java — HikariCP Connection Pool

```
// — File: DatabaseConfig.java — Real Infrastructure Example —————
@Configuration
public class DatabaseConfig {

    @Value("${spring.datasource.url}")
    private String url;

    @Value("${spring.datasource.username}")
    private String username;

    @Value("${spring.datasource.password}")
    private String password;

    @Bean
    @Primary // preferred bean when multiple DataSources are registered
    public DataSource dataSource() {
        HikariConfig cfg = new HikariConfig();
        cfg.setJdbcUrl(url);
        cfg.setUsername(username);
        cfg.setPassword(password);
        cfg.setMaximumPoolSize(10);
    }
}
```

```

        cfg.setMinimumIdle(2);
        cfg.setConnectionTimeout(30_000);
        cfg.setIdleTimeout(600_000);
        return new HikariDataSource(cfg);
    }
}

// — Corresponding application.yml —————
// spring:
//   datasource:
//     url:      jdbc:postgresql://localhost:5432/shopflow_users
//     username: ${DB_USERNAME}    # from environment variable
//     password: ${DB_PASSWORD}    # from environment variable

```



CGLIB Proxying in @Configuration

Spring subclasses @Configuration classes using CGLIB at runtime. This means that when you call passwordEncoder() inside another @Bean method, Spring intercepts the call and returns the existing singleton bean — it does NOT call BCryptPasswordEncoder(12) again. Without CGLIB, each call would create a new BCryptPasswordEncoder instance, defeating singleton scope.

Use @Configuration(proxyBeanMethods = false) to disable CGLIB when your @Bean methods never call each other — this gives a small performance improvement at startup.

Use @Bean When...	Use @Component/@Service When...
Class is from a third-party library (HikariCP, BCrypt)	Class is your own code that you can annotate
Construction requires multiple steps / complex logic	Construction is simple (just new + field assignment)
You need multiple beans of the same type	One bean per class is sufficient
Infrastructure config: DataSource, Kafka, Redis, JWT	Business logic: Services, Repositories, Controllers

5.3 Annotation-Based Configuration — Component Scanning

Component scanning is the dominant modern approach. You annotate your own classes with stereotype annotations, enable scanning, and Spring automatically discovers, registers, and wires them. No XML, no explicit @Bean method needed.

5.3.1 Enabling Component Scanning



Enabling Component Scanning

```

// — Option A: @ComponentScan on a @Configuration class —————
@Configuration
@ComponentScan(basePackages = "com.shopflow.user")
public class AppConfig {
    // Spring scans com.shopflow.user.** for @Component, @Service, etc.
}

// — Option B: @SpringBootApplication (recommended) —————

```

```
// @SpringBootApplication is a convenience annotation combining THREE annotations:
// @Configuration      - this class can define @Bean methods
// @EnableAutoConfiguration - Boot reads META-INF/spring/AutoConfiguration.imports
//                        and applies conditional auto-config (DataSource, MVC,
//                        etc.)
// @ComponentScan       - scans the package of this class + all sub-packages

@SpringBootApplication // = @Configuration + @EnableAutoConfiguration +
@ComponentScan
public class ShopflowUserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ShopflowUserServiceApplication.class, args);
    }
}

// ⚠ IMPORTANT: Place this class in the ROOT package
// com.shopflow.user.ShopflowUserServiceApplication ✓
// Sub-packages scanned automatically:
// com.shopflow.user.service.*    ← @Service beans found here
// com.shopflow.user.repository.* ← @Repository beans found here
// com.shopflow.user.controller.* ← @RestController beans found here
// com.shopflow.user.config.*     ← @Configuration beans found here
```

6. Stereotype Annotations — @Component and Its Specialisations

Spring provides @Component as the base stereotype annotation. Three specialisations carry additional semantic meaning and in some cases additional behaviour. Always use the most specific annotation available.

Annotation	Layer	Additional Behaviour Beyond @Component
@Component	Generic / Utility	None — use when no specialised annotation fits
@Service	Business / Service	None yet — semantic only, reserved for future Spring capabilities
@Repository	Data Access / Persistence	Exception Translation: converts DB-specific exceptions (SQLException, JPA PersistenceException) → Spring DataAccessException hierarchy
@Controller	Presentation (MVC)	Enables Spring MVC view resolution via @RequestMapping methods
@RestController	Presentation (REST API)	@Controller + @ResponseBody: every method return is serialised to JSON/XML response body
@Configuration	Config / Infrastructure	CGLIB subclassed — @Bean method calls return cached singletons

6.1 Stereotype Annotations — ShopFlow Examples

Stereotype Annotations in ShopFlow

```
// — @Service — Business Logic Layer —————
// Bean name (default): 'userService' (camelCase of class name)
@Service
public class UserService {
    // Business rules, orchestration, transaction boundaries
}

// Custom bean name:
@Service("userRegistrationService")
public class UserService { } // bean name = 'userRegistrationService'

// — @Repository — Data Access Layer —————
// Exception translation: SQLException → DataAccessException (auto by Spring)
@Repository
public class UserRepositoryImpl implements UserRepository {
    // JPA / JDBC operations
    // Any SQLException thrown here is caught and wrapped in DataAccessException
}

// — @RestController — REST API Layer —————
// = @Controller + @ResponseBody
// All method return values are written directly to HTTP response body as JSON
@RestController
@RequestMapping("/api/v1/users")
public class UserController {
    @GetMapping("/{id}")
    public UserResponse getUser(@PathVariable String id) {
        return userService.findById(id); // serialised to JSON automatically
    }
}
```

```
}  
}  
  
// — @Component — Generic Utility —————  
// Use when no specialised stereotype fits  
@Component  
public class RequestIdGenerator {  
    public String generate() { return UUID.randomUUID().toString(); }  
}
```

? Interview Question: What is the difference between @Component, @Service, and @Repository? Can I use @Component for everything?

✓ Technically yes — all three register a Spring bean identically. But you should not use @Component for everything for two reasons: (1) Semantic clarity — @Service communicates 'this is a business layer class', @Repository communicates 'this is a data access class'. Future developers and code analysis tools understand the architecture immediately. (2) Functional difference — @Repository enables Spring's exception translation: it wraps native DB exceptions (SQLException, JPA's PersistenceException) in Spring's DataAccessException hierarchy, giving you a consistent exception model regardless of the underlying database technology. Best practice: always use the most specific stereotype available.

7. Dependency Injection — All Three Types in Depth

Dependency Injection is Spring's mechanism for implementing IoC. Spring supports three injection types. The industry has a clear recommendation on which to use — and why — which is also a very common interview question.

7.1 Constructor Injection — The Recommended Approach

Dependencies are provided through the constructor. Spring calls the constructor with the appropriate beans when creating the bean. This is the recommended approach for all mandatory dependencies.

UserService.java — Constructor Injection

```
// — Constructor Injection - Full ShopFlow Example —————
@Service
public class UserService {

    // 'final' keyword enforces immutability - deps cannot change after
    construction
    private final UserRepository    userRepository;
    private final PasswordEncoder passwordEncoder;
    private final EmailService     emailService;
    private final AuditService     auditService;

    // Spring 4.3+: single constructor is auto-wired - no @Autowired needed
    public UserService(UserRepository userRepository,
                      PasswordEncoder passwordEncoder,
                      EmailService emailService,
                      AuditService auditService) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.emailService = emailService;
        this.auditService = auditService;
    }
    // In unit tests: new UserService(mockRepo, mockEncoder, mockEmail, mockAudit)
    // No Spring context needed - pure Java instantiation
}

// — With Lombok (eliminates constructor boilerplate) —————
// @RequiredArgsConstructor generates a constructor for ALL final fields
// Identical to writing the constructor manually above
@Service
@RequiredArgsConstructor // Lombok: generates constructor for all final fields
public class UserService {
    private final UserRepository    userRepository;
    private final PasswordEncoder passwordEncoder;
    private final EmailService     emailService;
    private final AuditService     auditService;
    // No constructor written - Lombok generates it at compile time
    // Spring sees ONE constructor → auto-wires without @Autowired
}
```

✓ Why Constructor Injection Is Recommended — 5 Reasons

1. Immutability: Dependencies are final — cannot be accidentally reassigned after construction.
2. Mandatory deps enforced: Missing bean = startup fails with clear error, not runtime NPE.

3. Testability: new UserService(mockRepo, mockEncoder...) works without Spring context at all.
4. Circular dep detection: Spring immediately reports BeanCurrentlyInCreationException — not silently proxied.
5. No reflection: Uses normal Java constructors — no magic, no hidden behaviour.

7.2 Setter Injection — For Optional Dependencies

Dependencies are injected through setter methods after the bean is constructed. Spring calls the setter(s) after instantiation. Use only for optional dependencies that have a reasonable default.

NotificationService.java — Setter Injection

```
// — Setter Injection — Use only for optional dependencies —
@Service
public class NotificationService {

    private EmailService emailService; // mandatory
    private SmsService smsService; // optional — not all environments have SMS

    // Required setter — Spring calls this; fails if no EmailService bean exists
    @Autowired
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }

    // Optional setter — required=false: smsService will be null if no bean found
    // Acceptable because email-only notification is still a valid state
    @Autowired(required = false)
    public void setSmsService(SmsService smsService) {
        this.smsService = smsService;
    }

    public void notify(String userId, String message) {
        emailService.send(userId, message);
        if (smsService != null) { // must guard against null optional dep
            smsService.send(userId, message);
        }
    }
}
```

7.3 Field Injection — @Autowired on Fields (Anti-Pattern)

Spring uses Java reflection to inject values directly into private fields. This is the most concise but most problematic approach. Understand it (you will see it in legacy code) but do not write it in new code.

Field Injection — Anti-Pattern (for recognition only)

```
// — Field Injection — What NOT to do in production —
@Service
public class OrderService {

    @Autowired
    private OrderService orderService;
```

```

private OrderRepository orderRepository; // Spring uses reflection to set
this

@Autowired
private InventoryClient inventoryClient;

@Autowired
private KafkaProducer kafkaProducer;
}

// — Why this is a problem in unit tests —————
class OrderServiceTest {
    @Test
    void testCreateOrder() {
        // Cannot do this — orderRepository is private with no constructor param:
        // OrderService service = new OrderService();
        // service.orderRepository = mockRepo; // private field!

        // You are FORCED to either:
        // A) Load a full Spring context (@SpringBootTest) — slow, expensive
        // B) Use ReflectionTestUtils.setField(service, 'orderRepository', mock)
        //     — brittle, breaks on field rename
        // Constructor injection eliminates this problem entirely.
    }
}

```

Criterion	Constructor Injection ✓	Setter Injection ⚠	Field Injection ✗
Recommended	✓ Industry standard	⚠ Only for optional deps	✗ Avoid in new code
Testable	✓ new Service(mock...)	⚠ Requires setter call	✗ Needs Spring or reflection
Immutable (final)	✓ Fields can be final	✗ Cannot be final	✗ Cannot be final
Clear dependencies	✓ Visible in constructor	⚠ Scattered in setters	✗ Hidden inside class body
Missing dep detection	✓ Startup failure, clear message	⚠ Startup failure if required=true	✗ NullPointerException at runtime
Circular dep detection	✓ Immediate BeanCurrentlyInCreation	⚠ May silently resolve via proxy	✗ May silently resolve via proxy

7.4 @Autowired, @Qualifier, and @Primary — Resolving Ambiguity

When Spring finds multiple beans of the same type and cannot determine which to inject, it throws `NoUniqueBeanDefinitionException`. Three mechanisms resolve this.

Resolving Bean Ambiguity — @Primary and @Qualifier

```

// — Scenario: Two EmailService implementations registered —————
@Service
@Primary // When ambiguity exists, Spring prefers this bean automatically
public class SendGridEmailService implements EmailService { ... }

```

```

@Service
@Profile("!prod") // Only active outside prod - avoids dual-registration
public class ConsoleEmailService implements EmailService { ... }

// — @Qualifier — specify exact bean by name —————
@Service
public class UserService {
    private final EmailService emailService;

    @Autowired
    public UserService(
        @Qualifier("sendGridEmailService") EmailService emailService,
        UserRepository userRepository) {
        this.emailService = emailService;
    }
}

```

? Interview Question: Why is field injection considered bad practice? What should you use instead?

✓ Three reasons: (1) Testability — you cannot instantiate the class with 'new' and pass mocks; you need Spring context or brittle ReflectionTestUtils. (2) Immutability — fields cannot be final, so dependencies could theoretically change after construction. (3) Hidden dependencies — the constructor (the class's public contract) does not reveal dependencies; you must read the class body. Solution: use constructor injection. With Lombok `@RequiredArgsConstructor` there is zero additional boilerplate — it generates the constructor for all final fields at compile time. IntelliJ IDEA flags field injection with a 'Field injection is not recommended' warning for exactly these reasons.

8. Bean Scopes — Controlling Instance Creation

Bean scope controls how many instances of a bean the container creates and how they are shared. Getting scopes wrong causes subtle correctness bugs (shared mutable state) or memory issues (too many instances). This is a heavily tested interview area.

8.1 Singleton Scope — One Instance Per ApplicationContext (Default)

Spring creates exactly ONE instance of the bean per ApplicationContext. Every class that injects this bean receives the same object reference. This is the default — you do not need to declare it explicitly.

Singleton Scope

```
// — Singleton (default) —  
@Service // scope = singleton by default  
public class UserService {  
    // One instance shared across ALL controllers, tests, other services  
}  
  
// Explicit declaration (equivalent — rarely written)  
@Service  
@Scope(ConfigurableBeanFactory.SCOPE_SINGLETON)  
public class UserService { }  
  
// — Proof: singleton returns same reference —  
@SpringBootTest  
class SingletonTest {  
    @Autowired ApplicationContext ctx;  
  
    @Test  
    void sameInstance() {  
        UserService a = ctx.getBean(UserService.class);  
        UserService b = ctx.getBean(UserService.class);  
        assertEquals(a, b); // PASSES — identical object reference  
    }  
}
```

⚠ CRITICAL — Singleton Beans Must Be Stateless

ONE singleton instance serves ALL threads simultaneously. Any mutable instance variable creates a race condition that is extremely hard to debug in production.

✗ WRONG — shared mutable state (race condition):

```
@Service public class OrderService {  
    private String currentOrderId; // shared across ALL threads!  
    private List<Item> tempItems; // one thread's data leaks to another!  
}
```

✓ CORRECT — stateless singleton (all state in method-local variables):

```
@Service  
@RequiredArgsConstructor  
public class OrderService {
```

```

private final OrderRepository repo; // injected, immutable, thread-safe
// All working data lives in method parameters or local variables
public Order create(CreateOrderRequest req) {
    List<Item> items = new ArrayList<>(); // LOCAL — each thread has its own
}
}

```

8.2 Prototype Scope — New Instance Every Request

Spring creates a new instance every time the bean is requested — via injection, `getBean()`, or `ObjectProvider`. Spring manages creation but NOT destruction of prototype beans.

Prototype Scope — CartBuilder Example

```

// — Prototype: ShopFlow CartBuilder —————
// Each cart should start fresh — prototype is the right choice
@Component
@Scope("prototype")
public class CartBuilder {
    private final List<CartItem> items = new ArrayList<>();
    private String userId;

    public CartBuilder forUser(String userId) { this.userId = userId; return this; }

    public CartBuilder add(String productId, int qty) {
        items.add(new CartItem(productId, qty)); return this;
    }

    public Cart build() { return new Cart(userId, List.copyOf(items)); }
}

// — WRONG: injecting prototype into singleton via @Autowired —————
// Spring injects CartBuilder ONCE at construction — defeats prototype scope!
@Service
public class OrderService {
    @Autowired
    private CartBuilder cartBuilder; // ❌ always the same instance
}

// — CORRECT Option 1: ObjectProvider —————
@Service
@RequiredArgsConstructor
public class OrderService {
    private final ObjectProvider<CartBuilder> cartBuilderProvider;

    public Cart buildCart(String userId, List<CartItemRequest> items) {
        CartBuilder builder = cartBuilderProvider.getObject(); // NEW each time
        builder.forUser(userId);
        items.forEach(i -> builder.add(i.productId(), i.qty()));
        return builder.build();
    }
}

// — CORRECT Option 2: @Lookup (Spring method injection) —————
@Service
public abstract class OrderService {
    @Lookup // Spring overrides this method to return a new prototype each call
    public abstract CartBuilder newCartBuilder();
}

```

```

public Cart buildCart(String userId, List<CartItemRequest> items) {
    CartBuilder builder = newCartBuilder(); // NEW instance every call
    builder.forUser(userId);
    return builder.build();
}
}

```

8.3 Web Scopes — request, session, application

These scopes are only available in a web-aware `ApplicationContext`. They tie a bean's lifecycle to a web concept. Each requires a scoped proxy so that the singleton that holds a reference to them always delegates to the correct instance.

Request and Session Scope

```

// — Request Scope: ONE instance per HTTP request —————
// Use case: per-request metadata — request ID, authenticated user from JWT
@Component
@RequestScope // = @Scope(value="request", proxyMode=ScopedProxyMode.TARGET_CLASS)
public class RequestContext {
    private final String requestId = UUID.randomUUID().toString();
    private String authenticatedUserId;

    public String getRequestId() { return requestId; }
    public void setAuthenticatedUserId(String id) { this.authenticatedUserId = id; }
}

// — Session Scope: ONE instance per HTTP session —————
// Use case: shopping cart persists across multiple requests from same user
@Component
@SessionScope
public class UserSessionCart implements Serializable {
    private final Map<String, Integer> items = new LinkedHashMap<>();

    public void addItem(String productId, int qty) {
        items.merge(productId, qty, Integer::sum);
    }
    public Map<String, Integer> getItems() {
        return Collections.unmodifiableMap(items);
    }
}

```

Scope	Instance Per	Use Case	Proxy Needed
singleton	ApplicationContext (entire app)	Stateless services, repos, configs	No
prototype	Each <code>getBean()</code> / injection point	Builders, stateful objects, commands	No (use <code>ObjectProvider</code>)
request	Each HTTP request	Request ID, per-request auth context	Yes — <code>TARGET_CLASS</code>
session	Each HTTP session (user session)	Shopping cart, user preferences	Yes — <code>TARGET_CLASS</code>

Scope	Instance Per	Use Case	Proxy Needed
application	ServletContext (entire web app)	App-wide read-only settings	Yes — TARGET_CLASS

? Interview Question: What is singleton scope in Spring? How is it different from the Singleton design pattern?

✓ In the Singleton design pattern, ONE instance exists per JVM/ClassLoader — enforced by the class itself (private constructor + static getInstance()). In Spring singleton scope, ONE instance exists per ApplicationContext — enforced by the container. Key differences: (1) You can have multiple ApplicationContexts in one JVM (common in tests), each with its own 'singleton' instance. (2) Spring singletons are plain Java classes — no private constructor, no getInstance(). (3) The container manages uniqueness, not the class. Singleton scope does NOT mean the class is itself a Singleton pattern.

9. Bean Lifecycle — From Creation to Destruction

Spring manages the complete lifecycle of every singleton bean. Understanding this lifecycle tells you exactly when initialisation callbacks fire, where AOP proxies are created, and how to perform clean resource cleanup.

9.1 Complete Lifecycle — 12-Step Sequence

A Spring singleton bean passes through these phases in exact order:

Bean Lifecycle — All 12 Steps

```
// ——— COMPLETE BEAN LIFECYCLE (12 steps) ———

// STEP 1: Bean definition loaded
//          Spring reads @Component/@Bean/XML and registers bean metadata

// STEP 2: Constructor called
//          Bean is instantiated (or factory method called)
//          Constructor-injected deps are resolved here

// STEP 3: Dependencies injected
//          @Autowired setters and fields populated

// STEP 4: Aware interfaces called (in order):
//          → BeanNameAware.setBeanName(name)
//          → BeanFactoryAware.setBeanFactory(factory)
//          → ApplicationContextAware.setApplicationContext(ctx)

// STEP 5: BeanPostProcessor.postProcessBeforeInitialization()
//          All registered BeanPostProcessors called BEFORE init

// STEP 6: @PostConstruct method called
//          Your initialisation code runs here

// STEP 7: InitializingBean.afterPropertiesSet() (if interface implemented)

// STEP 8: Custom init-method (if @Bean(initMethod='x') or XML init-method='x')

// STEP 9: BeanPostProcessor.postProcessAfterInitialization()
//          ← AOP PROXIES CREATED HERE (@Transactional, @Async, @Cacheable)

// STEP 10: Bean is READY — published for use in the application

// STEP 11: @PreDestroy method called (on context.close() / graceful shutdown)

// STEP 12: DisposableBean.destroy() and custom destroy-method called
```



Where Are AOP Proxies Created?

Step 9 — `BeanPostProcessor.postProcessAfterInitialization()` — is where Spring wraps your bean in a proxy. This is how `@Transactional`, `@Async`, `@Cacheable`, `@Secured`, and all AOP features work.

The proxy intercepts method calls and adds the cross-cutting behaviour (begin transaction, submit to thread pool, check cache, etc.) before and/or after delegating to your actual bean method.

This is why 'self-invocation' doesn't work with `@Transactional`: when your bean calls its own method internally, it bypasses the proxy — the proxy never sees the call.

9.2 `@PostConstruct` — Initialisation Callbacks

`@PostConstruct` marks a method to run after the bean is fully constructed and all dependencies are injected — but before the bean is made available to the rest of the application. It is the standard place for one-time startup logic.

`@PostConstruct` — Cache Warmup and DB Health Check

```
// — File: ProductCacheService.java — Cache Warmup —————
@Service
@RequiredArgsConstructor
@Slf4j
public class ProductCacheService {

    private final ProductRepository productRepository;
    private final RedisTemplate<String, Product> redisTemplate;

    // — @PostConstruct: runs AFTER constructor + dependency injection —
    // Runs BEFORE: this bean is handed to any controller or other service
    @PostConstruct
    public void warmUpCache() {
        log.info("[STARTUP] Warming product cache...");
        List<Product> products =
productRepository.findByStatus(ProductStatus.ACTIVE);
        products.forEach(p ->
            redisTemplate.opsForValue().set("product:" + p.getId(), p,
Duration.ofHours(1))
        );
        log.info("[STARTUP] {} products cached.", products.size());
    }

    // — @PreDestroy: runs when context is shutting down —————
    @PreDestroy
    public void evictCache() {
        log.info("[SHUTDOWN] Evicting local product cache.");
        // Redis keys expire via TTL — no manual cleanup needed here
        // But clear any in-memory structures:
        // localMap.clear();
    }
}

// — File: DatabaseHealthChecker.java — Fail-Fast on Bad Config —————
@Component
@RequiredArgsConstructor
@Slf4j
public class DatabaseHealthChecker {

    private final DataSource dataSource;

    @PostConstruct
    public void validateConnection() {
        try (Connection conn = dataSource.getConnection()) {
            if (!conn.isValid(5)) {
```

```

        throw new IllegalStateException("DB connection invalid at
startup!");
    }
    log.info("Database connection OK.");
} catch (Exception e) {
    // Fail fast — if DB is unreachable, don't start the service
    throw new IllegalStateException("FATAL: Cannot reach database", e);
}
}
}

```

Callback	When Called	Use For
@PostConstruct	After all deps injected, before bean published	Cache warmup, DB validation, Kafka topic creation, config loading
@PreDestroy	On context.close() / graceful shutdown (SIGTERM)	Close connections, flush logs, clear caches, release locks
InitializingBean.afterPropertiesSet()	Same time as @PostConstruct, called after it	Framework internals — prefer @PostConstruct in your code
DisposableBean.destroy()	Same time as @PreDestroy, called after it	Framework internals — prefer @PreDestroy in your code
@Bean(initMethod='x')	Same as @PostConstruct — for third-party classes	When you cannot modify the class source
@Bean(destroyMethod='y')	Same as @PreDestroy — for third-party classes	Closing third-party resources (e.g. Jedis pool)

? Interview Question: What is the difference between @PostConstruct and afterPropertiesSet()? Which should you use?

✓ Both run after all dependencies are injected, before the bean is published. Execution order: @PostConstruct fires FIRST, then afterPropertiesSet(). The key difference: @PostConstruct is a standard Jakarta/JSR-250 annotation — not Spring-specific. Your class does not need to implement any interface. afterPropertiesSet() is from Spring's InitializingBean interface — implementing it couples your class to Spring. Always prefer @PostConstruct. Use @Bean(initMethod='x') when you cannot modify the class (third-party library). Use afterPropertiesSet() only when studying Spring framework internals.

10. Conditional Beans — @Profile, @Conditional, @ConditionalOnProperty

Conditional beans are registered only when a specific condition is true — an active profile, a property value, a class on the classpath, etc. This is how Spring Boot's entire auto-configuration system works.

10.1 @Profile — Environment-Based Bean Switching

@Profile registers a bean only when a named profile is active. In ShopFlow, we use profiles to swap email providers, databases, and logging configurations between dev, test, and prod environments.

Profile-Based Bean Switching

```
// — Profile-based EmailService switching —————
@Configuration
public class EmailConfig {

    @Bean
    @Profile("prod") // only in production
    public EmailService sendGridEmailService(
        @Value("${sendgrid.api-key}") String apiKey) {
        return new SendGridEmailService(apiKey);
    }

    @Bean
    @Profile({"dev", "test", "default"}) // all non-prod environments
    public EmailService consoleEmailService() {
        return new ConsoleEmailService(); // just logs — no real emails sent
    }
}

// — Activating profiles —————
// In application.yml:
// spring.profiles.active: dev

// Via environment variable (recommended for Docker/K8s):
// SPRING_PROFILES_ACTIVE=prod

// Via JVM arg (for local testing):
// java -jar user-service.jar -Dspring.profiles.active=staging
```

10.2 @ConditionalOnProperty — Feature Flags

@ConditionalOnProperty — Audit Feature Flag

```
// — Feature flag: enable/disable audit trail via config property —————
@Bean
@ConditionalOnProperty(
    name = "features.audit.enabled",
    havingValue = "true",
    matchIfMissing = false // don't register if property is missing
)
public AuditService databaseAuditService(AuditRepository repo) {
    return new DatabaseAuditService(repo);
}
```

```
@Bean
@ConditionalOnProperty(
    name          = "features.audit.enabled",
    havingValue    = "false",
    matchIfMissing = true    // register this if property is absent
)
public AuditService noOpAuditService() {
    return event -> {};    // lambda no-op - does nothing
}

// — application.yml toggle (no code change, no redeployment) —————
// features:
//   audit:
//     enabled: true
```

11. Spring Application Events — In-Process Loose Coupling

Spring has a built-in event system for loose coupling within a single service. One bean publishes an event without knowing who listens. Other beans listen and react independently. This is the in-process equivalent of Kafka for within-service communication.

Application Events — UserRegisteredEvent

```
// — 1. Define the event (plain POJO — Spring 4.2+) —————
// Java record is perfect: concise, immutable
public record UserRegisteredEvent(String userId, String email, String firstName) {}

// — 2. Publish from UserService —————
@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository      userRepository;
    private final PasswordEncoder     passwordEncoder;
    private final ApplicationEventPublisher eventPublisher; // Spring built-in

    public UserResponse registerUser(CreateUserRequest req) {
        User saved = userRepository.save(buildUser(req));

        // Publish event — UserService does NOT know who listens
        // Decouples registration logic from email, audit, analytics
        eventPublisher.publishEvent(
            new UserRegisteredEvent(saved.getId(), saved.getEmail(),
            saved.getFirstName())
        );
        return UserResponse.fromEntity(saved);
    }
}

// — 3. Multiple independent listeners —————
@Component
@RequiredArgsConstructor
public class UserEventListeners {

    private final EmailService emailService;
    private final AuditService auditService;

    @EventListener
    public void sendWelcomeEmail(UserRegisteredEvent event) {
        emailService.sendWelcome(event.email(), event.firstName());
    }

    @EventListener
    public void logAuditTrail(UserRegisteredEvent event) {
        auditService.log("USER_REGISTERED", event.userId());
    }

    // @Async listener — runs in separate thread pool, does not block publisher
    // Requires @EnableAsync on a @Configuration class
    @EventListener
    @Async
    public void forwardToKafka(UserRegisteredEvent event) {
        // Forward to Kafka for cross-service notification (Phase 5)
    }
}
```

Spring Events vs Kafka — When to Use Which

Spring Application Events → IN-PROCESS (same JVM, same service).

Use for: decoupling beans within a single microservice.

Example: UserService → EmailService, AuditService (both in user-service).

Apache Kafka → CROSS-SERVICE (different JVMs, different microservices).

Use for: microservice-to-microservice async communication, guaranteed delivery.

Example: Order Service → Notification Service (different deployments).

In ShopFlow: use Spring Events within a service. Use Kafka between services. Both are needed.

12. Full Project Lab — ShopFlow User Service Foundation

Bring all concepts together by building the complete foundation of the ShopFlow User Service. Each class in this lab directly exercises concepts from this module. Follow each step — they build on each other.

12.1 Project Structure

Project Directory Structure

```
shopflow-user-service/
├── src/main/java/com/shopflow/user/
│   ├── ShopflowUserServiceApplication.java    // @SpringBootApplication
│   ├── config/
│   │   ├── AppConfig.java                    // @Configuration: @Bean definitions
│   │   └── EmailConfig.java                  // @Profile-based email switching
│   ├── controller/
│   │   └── UserController.java                // @RestController
│   ├── service/
│   │   ├── UserService.java                  // @Service — core business logic
│   │   ├── EmailService.java                 // interface
│   │   ├── ConsoleEmailService.java          // @Profile('dev') impl
│   │   └── SendGridEmailService.java         // @Profile('prod') impl
│   ├── repository/
│   │   └── UserRepository.java                // @Repository — Spring Data JPA
│   ├── domain/
│   │   └── User.java                          // @Entity
│   ├── dto/
│   │   ├── CreateUserRequest.java            // request DTO with validation
│   │   └── UserResponse.java                 // response DTO
│   └── event/
│       ├── UserRegisteredEvent.java          // plain record
│       └── UserEventListeners.java           // @EventListener methods
├── src/main/resources/
│   ├── application.yml                       // base config
│   ├── application-dev.yml                   // H2 in-memory DB for dev
│   └── application-prod.yml                  // PostgreSQL for prod
```

12.2 Key Classes

AppConfig.java

```
// — AppConfig.java — @Configuration infrastructure beans —
@Configuration
@EnableAsync
public class AppConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }

    @Bean(name = "asyncExecutor")
    public TaskExecutor taskExecutor() {
        ThreadPoolTaskExecutor ex = new ThreadPoolTaskExecutor();
```

```

        ex.setCorePoolSize(4);
        ex.setMaxPoolSize(10);
        ex.setThreadNamePrefix("shopflow-async-");
        ex.initialize();
        return ex;
    }
}

```

UserService.java — Complete

```

// ——— UserService.java — Applies: constructor injection, @Transactional, events
@Service
@RequiredArgsConstructor
@Slf4j
public class UserService {

    private final UserRepository      userRepository;
    private final PasswordEncoder     passwordEncoder;
    private final EmailService        emailService;
    private final ApplicationEventPublisher eventPublisher;

    @Transactional
    public UserResponse registerUser(CreateUserRequest req) {
        log.info("Registering: {}", req.getEmail());
        if (userRepository.existsByEmail(req.getEmail())) {
            throw new EmailAlreadyExistsException(req.getEmail());
        }
        User user = User.builder()
            .email(req.getEmail())
            .password(passwordEncoder.encode(req.getPassword()))
            .firstName(req.getFirstName())
            .lastName(req.getLastName())
            .build();
        User saved = userRepository.save(user);
        eventPublisher.publishEvent(
            new UserRegisteredEvent(saved.getId(), saved.getEmail(),
saved.getFirstName())
        );
        return UserResponse.fromEntity(saved);
    }

    @Transactional(readOnly = true)
    public UserResponse findById(String id) {
        return userRepository.findById(id)
            .map(UserResponse::fromEntity)
            .orElseThrow(() -> new UserNotFoundException(id));
    }
}

```

UserServiceTest.java — Unit Tests with Mocks

```

// ——— UserServiceTest.java — Pure unit test (no Spring context) ———
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock private UserRepository      userRepository;
    @Mock private PasswordEncoder     passwordEncoder;
    @Mock private EmailService        emailService;
    @Mock private ApplicationEventPublisher eventPublisher;

    // Works because UserService uses CONSTRUCTOR injection
}

```



```

@InjectMocks private UserService userService;

@Test
void register_happyPath_savesAndPublishesEvent() {
    var req = new CreateUserRequest("a@shopflow.com", "Pass@1", "Alice", "S");
    when(userRepository.existsByEmail(req.getEmail())).thenReturn(false);
    when(passwordEncoder.encode(req.getPassword())).thenReturn("hashed");
    User saved = User.builder().id("u1").email(req.getEmail())
        .firstName("Alice").password("hashed").build();
    when(userRepository.save(any())).thenReturn(saved);

    UserResponse res = userService.registerUser(req);

    assertEquals("u1", res.getId());
    verify(userRepository).save(any(User.class));
    verify(eventPublisher).publishEvent(any(UserRegisteredEvent.class));
}

@Test
void register_duplicateEmail_throwsException() {
    var req = new CreateUserRequest("dup@shopflow.com", "x", "B", "K");
    when(userRepository.existsByEmail(req.getEmail())).thenReturn(true);

    assertThrows(EmailAlreadyExistsException.class,
        () -> userService.registerUser(req));
    verify(userRepository, never()).save(any());
    verify(eventPublisher, never()).publishEvent(any());
}
}

```

🎯 Lab Exercise — 2 Hours

Scenario: Build the complete ShopFlow User Service foundation using all concepts from this module.

Step 1: Create project via Spring Initializr with: Web, JPA, H2, Validation, Actuator, Lombok. Step 2: Implement all classes above with the project structure shown. Step 3: Run with --spring.profiles.active=dev. Test via Postman: POST /api/v1/users. Step 4: Verify user saved — open H2 console at /h2-console. Step 5: Run UserServiceTest — all tests should pass without starting the server. Extension: Add a @RequestScope RequestContext bean that captures the request ID and authenticated user ID. Inject it into AuditService and log it with every audit entry.

13. Practical Architecture Pattern — Spring Core in Enterprise Systems

In a real enterprise microservice, Spring Core concepts are applied in a consistent, layered pattern. Every ShopFlow microservice will follow this structure.

13.1 Folder Structure and Annotation Mapping

Package / Folder	Annotation Used	Purpose
config/	@Configuration + @Bean	Infrastructure: DataSource, Kafka, Security, JWT, Redis
controller/	@RestController	HTTP entry points — parse request, call service, return response
service/	@Service + constructor injection	Business logic, transactions, event publishing
repository/	@Repository / JpaRepository	Data access — Spring Data JPA or JDBC
domain/	@Entity + JPA annotations	Domain model — maps to database tables
dto/	Plain Java records/classes	Request/Response objects — never expose @Entity directly
event/	Plain records + @EventListener	In-process events for loose coupling within the service

13.2 The Golden Rules of Spring Core Architecture

- Infrastructure beans (DataSource, Kafka, Security) → always @Bean in @Configuration
 - Business beans (UserService, OrderService) → always @Service with constructor injection
 - Data access beans → always @Repository with Spring Data JPA interface
 - Always program to interfaces (EmailService, not SendGridEmailService)
 - Singleton beans must be STATELESS — all mutable state in method-local variables
 - Use @PostConstruct for startup logic: cache warmup, DB validation, topic creation
 - Use @PreDestroy for shutdown: close connections, flush buffers, clear caches
 - Use Lombok @RequiredArgsConstructor — never write constructor boilerplate manually
 - Use @Profile to swap implementations for dev/test/prod — never if/else in code
-

14. Final Conceptual Summary

Spring Core provides the foundational infrastructure that every Spring microservice is built upon. Every feature — MVC, Security, JPA, Kafka, Actuator — sits on top of this core.

Spring Core Concept	What It Provides
IoC (Inversion of Control)	Design principle: delegate object creation to the container
Dependency Injection (DI)	Mechanism implementing IoC — Spring injects declared dependencies
BeanFactory	Basic IoC container — lazy initialisation, minimal features
ApplicationContext	Enterprise container — eager, events, i18n, AOP, profiles
XML Configuration	Legacy bean definitions in applicationContext.xml
Java @Configuration	Type-safe infrastructure bean definitions with @Bean
Annotation / Component Scan	Auto-discovery via @Component, @Service, @Repository
@SpringBootApplication	@Configuration + @EnableAutoConfiguration + @ComponentScan
Bean Scopes	Singleton (shared), Prototype (new per request), Request, Session
Bean Lifecycle	@PostConstruct (init) → AOP proxy creation → @PreDestroy (cleanup)
Constructor Injection	Recommended: immutable, testable, fail-fast on missing dep
Stereotype Annotations	@Service (business), @Repository (+exception translation), @RestController
@Profile / @Conditional	Conditional bean registration — dev vs prod implementations
ApplicationEvent System	In-process loose coupling between beans within a service

15. Top Interview Questions — Spring Core (20 Q&As)

These cover the most frequently asked Spring Core interview questions for freshers and junior developers entering the industry. Read each answer carefully — framing matters as much as the content.

? Interview Question: What is Spring Framework and why is it used?

✓ Spring is an open-source application framework that simplifies enterprise Java development by providing IoC, DI, AOP, data access, MVC, and security in an integrated manner. It is used because it eliminates boilerplate (no manual transaction management, connection management), promotes loose coupling through IoC, and integrates with every major Java technology. Spring Boot extends it further to enable production-ready microservices in minutes.

? Interview Question: What is the Spring IoC Container? What does it do?

✓ The IoC Container is the core engine of Spring. It reads configuration metadata (XML, `@Configuration`, annotations), creates beans based on that metadata, injects their dependencies, manages their lifecycle (init callbacks, AOP proxies, destroy callbacks), and manages scopes (singleton, prototype, request, session). `ApplicationContext` is the container implementation you always use — Spring Boot creates it automatically via `SpringApplication.run()`.

? Interview Question: What is a Spring Bean?

✓ A Spring Bean is any Java object whose lifecycle is managed by the Spring IoC container. It differs from a regular Java object in that: Spring creates it (not 'new' keyword), Spring injects its dependencies automatically, Spring calls lifecycle callbacks (`@PostConstruct`, `@PreDestroy`), and Spring applies AOP proxies (`@Transactional`, `@Async`). Beans are registered via `@Component/@Service/@Repository`, `@Bean` methods in `@Configuration`, or XML `<bean>` declarations.

? Interview Question: What does `@SpringBootApplication` do? What three annotations does it combine?

✓ `@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`.
(1) `@Configuration`: marks the class as a source of `@Bean` definitions. (2) `@EnableAutoConfiguration`: triggers Boot's auto-configuration — reads `META-INF/spring/AutoConfiguration.imports` and conditionally applies `DataSource` config, `MVC` config, security config, etc., based on what is on the classpath. (3) `@ComponentScan`: scans the package of the annotated class and all sub-packages for stereotype-annotated classes. This is why the main class must be in the root package.

? Interview Question: What happens if Spring cannot find a bean to inject?

✓ Spring throws `NoSuchBeanDefinitionException` at context startup time — not at runtime when a method is called. This is the 'fail-fast' advantage: misconfiguration is caught immediately on application startup rather than discovered in production hours later. If the dependency is optional, use `@Autowired(required = false)`, or `Optional<T>` as the injection type, or provide a `@ConditionalOnMissingBean` fallback bean.

? Interview Question: What is a circular dependency in Spring? How do you resolve it?

✓ A circular dependency occurs when Bean A depends on Bean B and Bean B depends on Bean A. With constructor injection, Spring detects this immediately at startup and throws `BeanCurrentlyInCreationException`. Resolution options: (1) Redesign — circular deps usually indicate a design smell; extract a third class C that both depend on. (2) `@Lazy` on one constructor parameter — Spring injects a proxy and breaks the cycle. (3) Use setter injection for one direction. Always prefer option 1 — circular deps are architectural red flags.

? **Interview Question: What is `@Primary` and `@Qualifier`? When do you use each?**

✓ `@Primary` marks one bean as the default when multiple beans of the same type exist. Spring automatically chooses the `@Primary` bean when no specific qualifier is requested. `@Qualifier('beanName')` at the injection point specifies exactly which bean to inject by name. Use `@Primary` when there is a clear 'default' implementation (e.g., `SendGridEmailService` in prod). Use `@Qualifier` when you need a specific non-default implementation at a particular injection point.

? **Interview Question: What is the difference between `@Bean` and `@Component`?**

✓ `@Component` is a class-level annotation — Spring auto-discovers it via component scanning. Use it on classes YOU own and can annotate. `@Bean` is a method-level annotation in a `@Configuration` class — the method creates and returns the bean. Use `@Bean` for: (1) third-party classes you cannot annotate (`HikariCP`, `BCrypt`), (2) beans needing complex construction logic, (3) multiple bean variants of the same type, (4) infrastructure components. The two are not mutually exclusive — enterprise projects use both.

? **Interview Question: Can you have multiple `@Configuration` classes? How does Spring merge them?**

✓ Yes — you can and should split configuration across multiple `@Configuration` classes (`AppConfig`, `DatabaseConfig`, `SecurityConfig`, `KafkaConfig`, etc.). Spring registers all of them and merges their `@Bean` definitions into a single `ApplicationContext`. Spring Boot auto-detects them all via `@ComponentScan`. Splitting config improves readability and separation of concerns — you can even put different `@Configuration` classes in different `@Profile` groups.

? **Interview Question: What is `@PostConstruct` and when is it called exactly?**

✓ `@PostConstruct` marks a method to run after: the bean's constructor has been called, all `@Autowired` dependencies have been injected, and all `Aware` interface callbacks have been called — but before the bean is published and made available to the rest of the application. Use it for: cache warmup, database connectivity validation, Kafka topic creation, loading reference data. Key point: if `@PostConstruct` throws an exception, the application fails to start — making misconfiguration visible immediately (fail-fast).

? **Interview Question: What is the thread-safety rule for singleton beans?**

✓ Singleton beans must be `STATELESS`. Because one singleton instance serves all threads concurrently, any mutable instance variable is shared between threads — creating race conditions. Stateless means: no instance variables except final injected dependencies and truly immutable constants. All working data lives in method-local variables (each thread gets its own stack frame). If you need per-user or per-request state, use request-scoped beans, session-scoped beans, or `ThreadLocal` — never mutable singleton state.

Module 2 Complete — Spring Core, IoC & Dependency Injection

Next: Module 3 — Spring MVC & REST API Development

ShopFlow Microservices Training Programme · Phase 2